Contents

Production-grade multilingual text-to-speech on NVIDIA DGX Spark
1. What this is
2. Commands cheat-sheet
3. Hardware and software targets
4. Repository layout
5. Installing the system
6. Running the system
7. Testing
8. Features
9. API reference
10. Calling backends directly (bypass the orchestrator)
11. Standalone single-model mode · integration guide
12. Use-case recipes
13. Troubleshooting
14. Operating guide
15. What's under the hood
16. Environment variables reference (.env)
17. Known limitations (v0.1.0)
18. Credits & references

Qwen3-TTS Studio

Production-grade multilingual text-to-speech on NVIDIA DGX Spark

Complete user & operator guide — version 0.1.0, April 2026

1. What this is

Qwen3-TTS Studio is a locally-hosted, production-grade text-to-speech stack that wraps Alibaba's **Qwen3-TTS-12Hz-1.7B** model family behind a single OpenAI-compatible HTTP API and a browser UI. It runs entirely on your **NVIDIA DGX Spark** (GB10 Grace-Blackwell), three models loaded concurrently, with live streaming PCM audio playback in the browser.

The full stack:

Layer	Role
Browser SPA (vanilla HTML/CSS/JS)	User interface: 4 tabs for preset voices, designed voices, cloned voices, and voice library.
FastAPI orchestrator (port 8080)	Single public entrypoint. Serves the UI, routes API calls to the right backend, persists uploaded voices, translates between OpenAI and vLLM-Omni request shapes.
vLLM-Omni × 3 (ports 8091/8092/8093)	One inference server per task type — CustomVoice, VoiceDesign, Base — all three resident simultaneously.
Qwen3-TTS model weights (~13 GB total on disk)	1.7B Talker + Code2Wav codec, one set per task.
Supervisord	Process manager. Starts the four services, auto-restarts failures, aggregated logs.

Three things it does really well

1. **Preset voices** — 9 studio-quality speakers covering English, Chinese, Japanese, Korean, plus Beijing and Sichuan Chinese dialects. Optional natural-language instructions to control tone, emotion, accent, and speed.
2. **Voice design** — describe a voice in plain language ("a warm elderly male narrator with a slow, reassuring pace") and the model synthesizes a brand-new voice matching that description.
3. **Voice cloning** — upload 3–15 s of clean speech plus (optionally) a transcript, and generate arbitrary text in that voice. Clones can be named, saved to a persistent library, and reused indefinitely.

All three modes support 11 languages and three streaming modes (full WAV, streamed PCM over HTTP, streamed text-in via WebSocket).

---

2. Commands cheat-sheet

All commands below assume you are inside the project root (wherever you cloned `Qwen-TTS-Studio`).

2.1 One-time setup

```
bash scripts/00_install.sh              # venv, vLLM 0.19.1 aarch64, vllm-omni (patched), flash-attn (s
bash scripts/10_download_weights.sh     # fetches any missing model weight sets (idempotent)
```

2.2 Lifecycle (full stack: orchestrator + 3 backends)

```
bash scripts/start.sh                   # spins up supervisord + 4 services (orchestrator + 3 vllm-omni
bash scripts/stop.sh                    # clean shutdown (SIGTERM; SIGKILL after 15 s)
bash scripts/restart.sh                 # stop + start
bash scripts/status.sh                  # supervisord process table + orchestrator /health JSON
bash scripts/logs.sh                    # tail all 5 logs (Ctrl-C to stop tailing; services keep runnin
```

2.2b Standalone single-model mode (for embedding / integrations)

Launch ONE backend directly — no orchestrator, no UI, no supervisord. Ideal for embedding Qwen3-TTS in another app. Ctrl-C to stop cleanly.

```
bash scripts/run_customvoice.sh                      # preset speakers on :8091
bash scripts/run_voicedesign.sh                      # voice design on :8092
bash scripts/run_base.sh                             # voice clone on :8093

# Or generic form (pick task / port / bind address):
bash scripts/run_standalone.sh CustomVoice           # default :8091, 0.0.0.0
bash scripts/run_standalone.sh VoiceDesign 8100      # on :8100
bash scripts/run_standalone.sh Base 8200 127.0.0.1   # localhost-only bind
bash scripts/run_standalone.sh --help                # full usage
```

After `Uvicorn running on http://…` appears, call it like any OpenAI-compatible speech server — see §11 for full integration examples.

2.3 Verification & benchmarks

```
bash scripts/smoke_test.sh              # 12 endpoint checks; writes sample WAVs to logs/smoke/
bash scripts/benchmark.sh 10            # 10 serial CustomVoice + VoiceDesign requests; prints wall-tim
```

2.4 Per-process controls (via supervisord)

Run these when you need to touch just one service instead of restarting everything. Programs are in group `qwentts`.

```
.venv/bin/supervisorctl -c configs/supervisord.conf status                    # all processes
.venv/bin/supervisorctl -c configs/supervisord.conf restart qwentts:orchestrator
.venv/bin/supervisorctl -c configs/supervisord.conf restart qwentts:vllm_base
.venv/bin/supervisorctl -c configs/supervisord.conf stop    qwentts:vllm_voicedesign
.venv/bin/supervisorctl -c configs/supervisord.conf start   qwentts:vllm_voicedesign
.venv/bin/supervisorctl -c configs/supervisord.conf tail -f qwentts:vllm_customvoice
```

Program names: `orchestrator`, `vllm_customvoice`, `vllm_voicedesign`, `vllm_base`.

2.5 Per-model load / unload from the HTTP API

Same thing, but through the orchestrator. Good for automation and for the UI.

```
# List all models with status
curl -s http://127.0.0.1:8080/v1/admin/models | python3 -m json.tool

# Unload a single model to free its ~4.3 GB of VRAM
curl -X POST http://127.0.0.1:8080/v1/admin/models/VoiceDesign/unload

# Load it back (blocks until ready, ~30-60 s cold start)
curl -X POST "http://127.0.0.1:8080/v1/admin/models/VoiceDesign/load?wait=true"

# Fire-and-forget version
curl -X POST "http://127.0.0.1:8080/v1/admin/models/Base/load?wait=false"

# Unload everything (free all GPU memory)
curl -X POST http://127.0.0.1:8080/v1/admin/models/all/unload

# Load everything
curl -X POST http://127.0.0.1:8080/v1/admin/models/all/load
```

From the UI: the **Models** tab is the first one. Each of the 3 cards has a "Load" and "Unload" button, with live state pills. Bulk buttons are `▶ Load all` / `⏹ Unload all` / `⟳ Refresh`.

2.6 Health, info, metadata

```
curl -s http://127.0.0.1:8080/health              | python3 -m json.tool   # status + per-backend up/do
curl -s http://127.0.0.1:8080/info                | python3 -m json.tool   # full capabilities + models
curl -s http://127.0.0.1:8080/v1/tts/languages    | python3 -m json.tool   # the 11 supported languages
curl -s http://127.0.0.1:8080/v1/tts/tasks        | python3 -m json.tool   # per-task speaker lists + e
curl -s http://127.0.0.1:8080/v1/audio/voices     | python3 -m json.tool   # built-in + uploaded voices
```

2.7 Generating speech (orchestrator)

```
# Preset voice (CustomVoice)
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"CustomVoice","voice":"ryan","language":"English",
       "input":"Hello from the DGX Spark."}' --output out.wav

# Describe-a-voice (VoiceDesign)
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"VoiceDesign","language":"English",
       "instructions":"a calm, warm narrator",
       "input":"Once upon a time..."}' --output narrator.wav

# Voice cloning (Base) — inline reference audio
B64=$(base64 -w0 my_reference.wav)
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d "{\"task_type\":\"Base\",\"input\":\"Cloned voice speaking.\",
       \"ref_audio\":\"data:audio/wav;base64,$B64\",
       \"ref_text\":\"What you said in the reference clip.\"}" \
  --output cloned.wav

# Streaming PCM to speakers (real-time playback)
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"CustomVoice","voice":"vivian",
       "input":"Streaming audio!","stream":true,"response_format":"pcm"}' \
  --no-buffer | play -t raw -r 24000 -e signed -b 16 -c 1 -
```

2.8 Voice library (cloning with persistence)

```
# Upload a voice; persists across restarts
curl -sX POST http://127.0.0.1:8080/v1/audio/voices \
  -F "audio_sample=@my_voice.wav" \
  -F "consent=ui-$(date +%s)" \
  -F "name=my_narrator" \
  -F "ref_text=Exact transcript of my_voice.wav" \
  -F "speaker_description=warm narrator"

# Use the saved voice by name
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"Base","input":"Hello in my cloned voice.","ref_voice":"my_narrator"}' \
  --output me.wav

# Preview a saved voice's reference clip
curl -s http://127.0.0.1:8080/v1/audio/voices/my_narrator/preview --output preview.wav

# Delete a saved voice
curl -X DELETE http://127.0.0.1:8080/v1/audio/voices/my_narrator
```

2.9 Direct backend calls (bypass orchestrator)

```
curl -X POST http://127.0.0.1:8091/v1/audio/speech -H 'Content-Type: application/json' -d '{...}' # Cus
curl -X POST http://127.0.0.1:8092/v1/audio/speech -H 'Content-Type: application/json' -d '{...}' # Voi
curl -X POST http://127.0.0.1:8093/v1/audio/speech -H 'Content-Type: application/json' -d '{...}' # Bas
```

See §10 for when to hit backends directly vs. going through the orchestrator.

2.10 Diagnostics

```
nvidia-smi                                          # GPU + per-process memory
ps axf | grep vllm-omni                             # backend process tree
du -sh "$ROOT_DIR"/models/*                         # weights on disk
ls -lh logs/                                        # all service logs + sizes
grep -Ei 'error|traceback|fatal' logs/*.log | tail -20   # first place to look when something breaks
```

---

3. Hardware and software targets

Tested hardware: NVIDIA DGX Spark

- GB10 Grace-Blackwell Superchip (SM 12.1)
- 128 GB unified LPDDR5x @ 273 GB/s
- 20× ARM Cortex cores (10× X925 + 10× A725)
- aarch64 Linux, CUDA 13.0, Driver 580.95.05

Software stack: Python 3.12 · PyTorch 2.10+cu130 · vLLM 0.16.0+cu130 (aarch64 wheel) · vLLM-Omni (main) · Flash-Attention 2.8.3 (compiled from source) · Transformers 4.57.3

Memory budget (all 3 models resident):

Component	Approx VRAM
3× Talker-1.7B weights	10.5 GB
3× Code2Wav decoder	2.1 GB
KV caches (3 talkers + 3 codecs)	4.5 GB
vLLM/orchestrator overhead	3.5 GB
Total at idle	**~20.6 GB / 128 GB**

Plenty of headroom. Settings configured at `gpu_memory_utilization: 0.15` per stage (6 stages × 0.15 = 0.9 total utilisation ceiling).

---

4. Repository layout

Every file is under `$ROOT_DIR/` (the project root — whatever directory you cloned the repo into). Here is what each piece is for.

Top level

Path	What it is
`README.md`	Quick-start cheatsheet
`.env`	All runtime paths, ports, memory settings — edit here to retune
`pyproject.toml`	Python dependency manifest for the orchestrator
`.venv/`	Python virtual environment (created by `scripts/00_install.sh`)
`vllm-omni/`	Cloned + patched vLLM-Omni source, installed editable
`models/`	Model weights (~13 GB). Four subdirs, one per model
`docs/`	This document and its sources

Models (`models/`)

Directory	Role
`Qwen3-TTS-Tokenizer-12Hz/`	Shared speech codec (12 Hz, 16-codebook RVQ). Used by all three models.
`Qwen3-TTS-12Hz-1.7B-CustomVoice/`	9 preset speakers with optional instruction control.
`Qwen3-TTS-12Hz-1.7B-VoiceDesign/`	Designs a fresh voice from a natural-language description.
`Qwen3-TTS-12Hz-1.7B-Base/`	Voice cloning from a 3–15 s reference clip (plus optional transcript).

Configs (`configs/`)

File	Purpose
`qwen3_tts_dgx.yaml`	vLLM-Omni stage config tuned for GB10 + 3-model co-residency. Sets `gpu_memory_utilization: 0.15`, `max_num_seqs: 4` on the Talker, `enforce_eager: true` on the codec.
`supervisord.conf.tpl`	Process manager template. Expanded at start time via `envsubst` into `supervisord.conf`. Defines 4 programs: `vllm_customvoice`, `vllm_voicedesign`, `vllm_base`, `orchestrator`.

Scripts (`scripts/`)

Every script is idempotent and can be re-run safely.

Script	Action
`00_install.sh`	One-shot installer. Creates `.venv`, installs vLLM aarch64 wheel for CUDA 13, clones + patches `vllm-omni` (removes `fa3-fwd` which doesn't build on ARM64), compiles flash-attn from source, installs orchestrator Python deps. Takes ~30–60 min. Re-running skips steps already done.
`10_download_weights.sh`	Downloads any missing model weights via `hf download`. Base, CustomVoice, and Tokenizer are already in place; this only needs to fetch VoiceDesign on first run (~3.9 GB).
`start.sh`	Renders `supervisord.conf.tpl` → `supervisord.conf` with env vars substituted, then runs `supervisord` daemonised. All 4 services come up.
`stop.sh`	Graceful shutdown. Uses `supervisorctl shutdown`, falls back to `SIGKILL` after 15 s.
`restart.sh`	Stop + start.
`status.sh`	Lists supervisord process statuses and hits `/health` on the orchestrator.
`logs.sh`	`tail -Fn 20` of all 5 log files (orchestrator + 3 backends + supervisord). Ctrl-C to stop tailing.
`smoke_test.sh`	End-to-end curl-based sanity check of every public endpoint. Writes output WAVs to `logs/smoke/`.
`benchmark.sh [N]`	Runs N (default 5) sequential generations per task and reports per-run + average wall time.

Orchestrator (`orchestrator/`)

The FastAPI service at port 8080.

Module	Responsibility
`app.py`	FastAPI entrypoint + lifespan. On startup: probes backends, loads voice library, replays cloned voices to Base backend. On shutdown: stops probe loop, closes httpx client.
`config.py`	Pydantic-settings reading `.env`. Holds all paths/ports in a single `Settings` object.
`schemas.py`	Pydantic request/response models, `SpeechRequest`, `VoiceRef`, `VoicesList`, `HealthPayload`.
`backends.py`	`BackendRegistry` — the 3 vLLM-Omni backends. Runs a periodic probe loop (every 10 s) against `/v1/audio/voices` to check up/down.
`voice_library.py`	On-disk persistent store of user-uploaded voices. Survives restarts. Can replay uploads to a freshly-started Base backend so cloned voices are always available.
`audio.py`	MIME mapping and a tiny PCM16 → WAV wrapper helper.
`routes/system.py`	`GET /health`, `GET /info`, `GET /v1/tts/languages`, `GET /v1/tts/tasks`.
`routes/voices.py`	`GET /v1/audio/voices`, `POST /v1/audio/voices` (upload), `DELETE /v1/audio/voices/{name}`, `GET /v1/audio/voices/{name}/preview`.
`routes/speech.py`	`POST /v1/audio/speech` with OpenAI-compatible schema + Qwen3-TTS extensions. Task-type routing. Handles non-streaming (buffered) and streaming (chunked PCM) paths.
`routes/websocket.py`	`WS /v1/audio/speech/stream` — passthrough bridge to the corresponding vLLM-Omni WebSocket.

UI (`static/`)

No build step. No npm. Loads as ES modules via `<script type="module">`.

File	Purpose
`index.html`	Single-page shell with 4 tabs + toast notifications.
`css/reset.css`	Minimal CSS reset.
`css/app.css`	Full application styles. Auto light/dark via `prefers-color-scheme`.
`assets/logo.svg`	App logo (inline SVG gradient).
`js/utils.js`	`$`, `$$`, `toast`, `fmtBytes`, `fmtTime`, `fillSelect`, `debounce`.
`js/api.js`	Thin `fetch` wrapper over the orchestrator endpoints.
`js/tabs.js`	Tab switcher + segmented control switcher.
`js/pcm_player.js`	**Gapless WebAudio player for streamed 16-bit little-endian mono PCM.** Handles odd-byte alignment between fetch chunks. Accumulates bytes for a final download-as-WAV blob.
`js/voice_library.js`	Renders the Voice Library tab, syncs the library dropdown in Clone mode.
`js/app.js`	Main SPA controller. Wires all 4 tabs, health polling, streaming vs non-streaming, saving uploads.

Data (`data/`)

Path	Contents
`voices/`	Uploaded reference-audio WAV files.
`voices.json`	Catalog metadata: name, mime, file path, size, created_at, ref_text, speaker_description, language, consent_id.
`generated/`	Reserved for an optional TTL-expiring cache of generated audio (not used by default).

Logs (`logs/`)

One file per process. Rotated at 20 MB × 3 backups.

```
orchestrator.log    supervisord.log
customvoice.log     voicedesign.log    base.log
```

---

5. Installing the system

4.1 Prerequisites

- DGX Spark running Linux, CUDA 13.0, driver 580+.

- `uv` (Python package manager — already installed in your environment).
- `sudo` access once, for `apt-get install` of `ffmpeg sox python3.12-dev build-essential git ninja-build`.
- About 1.8 TB of free disk (you have 1.7 TB — fine).

4.2 Run the installer

```
cd /path/to/Qwen-TTS-Studio       # the directory you cloned into
bash scripts/00_install.sh
```

What it does, in order:

1. Creates `.venv` with Python 3.12 via `uv venv`.
2. Installs system build deps via `apt-get` (prompts for sudo once).
3. Installs the **vLLM 0.16.0 aarch64+cu130 wheel** (pinned URL, community-built, verified on GB10).
4. Clones `vllm-omni`, **removes the `fa3-fwd==0.0.2` dep** from `requirements/cuda.txt` (Hopper-only, won't build on Blackwell/ARM), and installs editable with `[demo]` extras.
5. Builds **flash-attention 2.8.3 from source** (`MAX_JOBS=4 NVCC_THREADS=2` to stay under memory ceilings). This is the longest step: **20–30 minutes** on GB10.
6. Installs the orchestrator's FastAPI/httpx/etc deps from `pyproject.toml`.
7. Runs an import smoke-test to confirm everything loads.

All output is both streamed to your terminal and logged to `logs/install.log`.

4.3 Download the VoiceDesign weights

Three of the four weight sets are already on disk (moved in-place during setup). The installer leaves VoiceDesign for this step because it's a fresh ~3.9 GB download.

```
bash scripts/10_download_weights.sh
```

The script is idempotent — re-running it will skip anything already present. Output directory after success:

```
models/Qwen3-TTS-Tokenizer-12Hz          (~651 MB)
models/Qwen3-TTS-12Hz-1.7B-Base          (~4.3 GB)
models/Qwen3-TTS-12Hz-1.7B-CustomVoice   (~4.3 GB)
models/Qwen3-TTS-12Hz-1.7B-VoiceDesign   (~4.3 GB)  ← new
```

---

6. Running the system

5.1 Start

```
bash scripts/start.sh
```

Four processes come up under `supervisord`:

```
qwentts:vllm_customvoice   STARTING  pid xxxxx
qwentts:vllm_voicedesign   STARTING  pid xxxxx
qwentts:vllm_base          STARTING  pid xxxxx
qwentts:orchestrator       RUNNING   pid xxxxx
```

First-run model loads take **30–90 seconds each**. During this time `/health` will return `degraded` (orchestrator up, some backends still loading). Once all three are `up` the overall state becomes `healthy`.

5.2 Observe

```
bash scripts/status.sh       # one-shot process table + health JSON
bash scripts/logs.sh         # continuous tail of all 5 logs
```

Open the UI:

```
http://127.0.0.1:8080/
```

(Orchestrator is bound to `127.0.0.1` by design. If you need to reach it from another machine, SSH-forward: `ssh -L 8080:127.0.0.1:8080 <dgx-spark-host>`.)

5.3 Stop / restart

```
bash scripts/stop.sh
bash scripts/restart.sh
```

`stop.sh` issues `supervisorctl shutdown`, waits up to 15 s for clean exit, then `SIGKILL`s. Voice library files in `data/` are untouched.

5.4 Re-read configuration

All ports, paths, and the per-stage memory utilisation live in `.env`. After editing:

```
bash scripts/restart.sh
```

7. Testing

6.1 Smoke test — run it after every install or weight change

```
bash scripts/smoke_test.sh
```

It hits **every public endpoint** and asserts a non-empty response:

1. `GET /health`
2. `GET /info`
3. `GET /v1/tts/languages`
4. `GET /v1/tts/tasks`
5. `GET /v1/audio/voices`
6. `POST /v1/audio/speech` — CustomVoice, preset voice (WAV)
7. `POST /v1/audio/speech` — VoiceDesign (WAV)
8. `POST /v1/audio/speech` — CustomVoice streaming PCM
9. `POST /v1/audio/voices` — upload the CustomVoice output as a fake reference
10. `POST /v1/audio/speech` — Base, using the uploaded library voice
11. `DELETE /v1/audio/voices/{name}` — cleanup

Generated audio is saved under `logs/smoke/` so you can listen and confirm by ear. Exit status 0 means every step passed.

6.2 Benchmark

```
bash scripts/benchmark.sh        # 5 runs per task, default
bash scripts/benchmark.sh 10     # 10 runs per task
```

Prints per-run and average wall-time for CustomVoice and VoiceDesign. Results saved to `logs/bench/`.

Expected baseline on GB10 (single user, ~100-char input):

Task	First-token latency	Total wall time
CustomVoice preset	200–300 ms	~1.0–1.4 s
VoiceDesign	250–400 ms	~1.2–1.6 s
Base (voice clone with 8 s ref)	500–800 ms	~1.6–2.2 s

6.3 Manual UI sanity

Open http://127.0.0.1:8080/ and verify:

- Top-right status cluster: three green dots (CustomVoice, VoiceDesign, Base).

- **Preset tab**: pick a speaker, type some text, click Generate — audio card appears, plays.
- **Design tab**: enter a voice description and text, click Generate.
- **Clone tab → Upload & clone**: drop a WAV file, enter transcript, generate. Optionally save to library.
- **Library tab**: saved voices appear, can be previewed, used in Clone, or deleted.
- Toggle "Stream as generated" off: the WAV is fetched whole and the audio control seeks/rewinds cleanly.

---

8. Features

7.1 Preset voices (CustomVoice)

Nine studio-quality speakers covering six native languages. Use the `voice` field in the API, or the speaker dropdown in the Preset tab.

Voice	Description	Native language
`vivian`	Bright, slightly edgy young female voice	Chinese
`serena`	Warm, gentle young female voice	Chinese
`uncle_fu`	Seasoned male, low mellow timbre	Chinese
`dylan`	Youthful Beijing male, clear natural timbre	Chinese (Beijing dialect)
`eric`	Lively Chengdu male, slightly husky brightness	Chinese (Sichuan dialect)
`ryan`	Dynamic male, strong rhythmic drive	English
`aiden`	Sunny American male, clear midrange	English
`ono_anna`	Playful Japanese female, light nimble timbre	Japanese
`sohee`	Warm Korean female, rich emotion	Korean

Every voice can speak any of the 11 supported languages, but quality is best in the voice's native language.

7.2 Voice design (VoiceDesign)

Describe a voice in natural language; the model designs a fresh voice matching that description. There are no predefined speakers in this mode — the `instructions` field is the voice description.

Examples of descriptions:

- "a warm elderly male narrator with a slow, reassuring pace"
- "an energetic young female podcast host"
- "a gravelly, weary detective in his sixties"

- "a crisp neutral newsreader"
- "体现撒娇稚嫩的萝莉女声，音调偏高" (playful young female, slightly high-pitched)

7.3 Voice cloning (Base)

Upload a 3–15 s clean reference clip; clone that voice and speak arbitrary text in it. Supply the transcript of the reference clip (in-context cloning, **ICL mode**) for best quality — without a transcript, only the speaker embedding is used (x-vector mode, lower fidelity).

Uploading a voice (two paths):

- **Ad-hoc** — in the Clone tab, drop a file and leave "Save to library as" empty. The ref audio is sent base64-inline in the generate request and not persisted.
- **Saved to library** — fill in a name. The audio is uploaded to the voice library (persisted on disk, mirrored to the Base backend, survives restarts), and subsequent clone requests just reference `ref_voice: <name>`.

The voice library is OpenAI-standard: `GET` lists, `POST` uploads, `DELETE` removes. Libraries survive orchestrator restarts because the voice data lives on disk under `data/voices/`.

7.4 Instructions / style / emotion

Every task accepts a free-form **`instructions`** field that modulates tone, emotion, accent, and speed. Required for VoiceDesign, optional for CustomVoice and Base. Examples that work across any speaker:

- "Speak with a British accent"
- "Speak with a Southern American accent"
- "Speak excitedly"
- "Speak quietly, as if telling a secret"
- "Speak slowly and professionally"
- "用开心的语气说" (speak in a happy tone — Chinese)
- "Speak angrily but controlled"

7.5 Languages

All three tasks support these 11 language codes in the **`language`** field:

```
Auto, Chinese, English, Japanese, Korean, German,
French, Russian, Portuguese, Spanish, Italian
```

`Auto` lets the model detect the language from the input text — usually fine; set explicitly if you know.

7.6 Streaming

Two orthogonal kinds of streaming:

A. Streaming PCM output (HTTP) — set `stream: true, response_format: "pcm"`. The server emits raw 16-bit little-endian mono PCM at 24 kHz, one chunk per Code2Wav window (25 frames, ~320 ms of audio), as soon as it's decoded. The browser UI (and our `pcm_player.js`) assembles these with gapless WebAudio scheduling, for sub-300 ms time-to-first-audio.

Constraints when `stream=true`: - Must use `response_format: "pcm"`. - `speed` adjustment is ignored. - Works on every task type.

B. Streaming text input (WebSocket) — `WS /v1/audio/speech/stream`. Send a `session.config` JSON frame, then one or more `input.text` frames, then `input.done`. The server buffers + splits at sentence boundaries and streams PCM audio per sentence. Useful for LLM+TTS pipelines where the text is arriving incrementally.

7.7 Output formats

`wav`, `mp3`, `flac`, `pcm`, `aac`, `opus`. `wav` is the default and has zero server-side overhead. The others are transcoded by the backend.

7.8 Voice library

Persistent catalog of uploaded reference voices. Backed by `data/voices.json`. API and UI cover:

- **List**: `GET /v1/audio/voices` returns all voices (built-in per task + uploaded).
- **Upload**: `POST /v1/audio/voices` with `multipart/form-data` (fields: `audio_sample`, `consent`, `name`, `ref_text?`, `speaker_description?`, `language?`). Max 10 MB, WAV/MP3/FLAC/OGG/AAC/WebM/MP4.
- **Preview**: `GET /v1/audio/voices/{name}/preview` streams the stored audio back.
- **Delete**: `DELETE /v1/audio/voices/{name}`.

Self-heal behaviour: the orchestrator keeps the source-of-truth copy of every voice on disk. If the Base vLLM-Omni backend restarts (OOM, kernel panic, manual restart), the orchestrator re-uploads every saved voice to it on its next up-probe, so the catalog is always in sync.

7.9 OpenAI drop-in

Because the orchestrator speaks `POST /v1/audio/speech`, any OpenAI client works unchanged:

```
from openai import OpenAI
client = OpenAI(base_url="http://127.0.0.1:8080/v1", api_key="none")
r = client.audio.speech.create(
    model="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice",
    voice="vivian",
    input="Hello from Qwen3-TTS!",
)
r.stream_to_file("out.wav")
```

You do not need an API key; the orchestrator is localhost-only and unauthenticated. Pass `api_key="none"` or any dummy string; the SDK requires something non-empty.

9. API reference

Base URL: `http://127.0.0.1:8080`.

8.1 System

Method	Path	Description
GET	`/health`	Aggregated status with per-backend detail.
GET	`/info`	Models + paths + status + supported languages/formats/features.
GET	`/v1/tts/languages`	Static list of language strings.
GET	`/v1/tts/tasks`	Per-task UI helper: speaker catalog, example instructions, tips.

`/health` response:

```
{
  "status": "healthy",
  "backends": {
    "customvoice": "up",
    "voicedesign": "up",
    "base": "up"
  },
  "uptime_s": 348
}
```

Statuses: `healthy` (all 3 up), `degraded` (≥ 1 up), `unhealthy` (0 up).

8.2 Voices

GET /v1/audio/voices

Returns merged list: built-in voices from each loaded backend, plus uploaded voices.

```
{
  "voices": ["aiden", "dylan", "eric", "my_narrator", "ono_anna", "ryan", "serena", "sohee", "uncle_fu"
  "uploaded_voices": [
    {
      "name": "my_narrator",
      "mime_type": "audio/wav",
      "file_path": "/home/.../data/voices/my_narrator.wav",
      "file_size": 482304,
      "created_at": 1745162401,
      "consent_id": "ui-1745162401",
      "ref_text": "The quick brown fox jumps over the lazy dog.",
      "speaker_description": "warm calm narrator",
      "language": "English"
    }
  ],
  "builtin_by_task": {
    "CustomVoice": ["aiden", "dylan", "eric", "ono_anna", "ryan", "serena", "sohee", "uncle_fu", "vivia
    "VoiceDesign": [],
    "Base": ["my_narrator"]
  }
}
```

POST /v1/audio/voices

Upload a new voice. `multipart/form-data`:

Field	Type	Required	Notes
`audio_sample`	file	yes	≤ 10 MB. wav / mp3 / flac / ogg / aac / webm / mp4
`consent`	string	yes	Any unique identifier; `ui-<timestamp>` is conventional
`name`	string	yes	Used as voice identifier and filename. No `/` or `..`
`ref_text`	string	no	Transcript of the reference. Enables ICL mode (higher quality)
`speaker_description`	string	no	Free-form description, returned in `GET /v1/audio/voices`
`language`	string	no	Native language of the reference clip

Response:

```
{ "success": true, "voice": { ... }, "mirrored_to_base_backend": true }
```

`mirrored_to_base_backend: false` means the orchestrator stored the voice but the Base backend was unavailable — it will be replayed automatically when the backend comes back up.

DELETE /v1/audio/voices/{name}

Removes the voice from the library (both disk and cache).

GET /v1/audio/voices/{name}/preview

Streams the reference audio back for UI previewing.

8.3 Speech generation

POST /v1/audio/speech

OpenAI-style body with Qwen3-TTS extensions:

```
{
  "input": "Text to synthesize",
  "task_type": "CustomVoice",
  "voice": "vivian",
  "language": "English",
  "instructions": "Speak excitedly",
  "response_format": "wav",
  "speed": 1.0,
  "max_new_tokens": 2048,
  "stream": false,

  "ref_audio": "... (Base only)",
  "ref_text": "... (Base only)",
  "ref_voice": "my_narrator (Base only, library reference)",
  "x_vector_only_mode": false
}
```

Routing: the orchestrator picks the backend from `task_type`. `ref_voice` resolves against the voice library and becomes `ref_audio` + `ref_text` automatically.

Response (non-streaming): binary audio body with matching `Content-Type`. Headers include:

```
X-Qwentts-Task: CustomVoice
X-Qwentts-Format: wav
```

Response (streaming): `Content-Type: audio/L16; rate=24000; channels=1` (raw PCM), one chunk per Code2Wav window. Append to a WebAudio context and play immediately, or collect + wrap in a WAV header for download.

8.4 Streaming WebSocket

`WS /v1/audio/speech/stream`

Message exchange:

```
// Client → server
{ "type": "session.config", "voice": "vivian", "task_type": "CustomVoice",
  "language": "English", "stream_audio": true, "response_format": "pcm" }
{ "type": "input.text", "text": "Hello, " }
{ "type": "input.text", "text": "how are you?" }
{ "type": "input.done" }

// Server → client (interleaved text + binary)
{ "type": "audio.start", "sentence_index": 0, "sentence_text": "Hello, how are you?",
  "format": "pcm", "sample_rate": 24000 }
/* one or more binary PCM frames */
{ "type": "audio.done", "sentence_index": 0, "total_bytes": 96000, "error": false }
{ "type": "session.done", "total_sentences": 1 }
```

10. Calling backends directly (bypass the orchestrator)

The stack has **four HTTP endpoints**, not one. Everyday use goes through the FastAPI orchestrator on **port 8080** — that's the simple, recommended path. But every vLLM-Omni backend is itself a fully-featured OpenAI-compatible speech server on its own port, and in some situations it's better to talk to one directly.

```
        :8080 ─── FastAPI orchestrator (you usually want this)
               │
               ├── routes by task_type to…
               │
  :8091 ─── CustomVoice backend  ── preset speakers (vivian, ryan, …)
  :8092 ─── VoiceDesign backend  ── prompt-described voices
  :8093 ─── Base backend         ── voice cloning from reference audio
```

9.1 Decision matrix — which port to hit

What you want to do	Use	Why
Build anything user-facing (UI, app)	**:8080 (orchestrator)**	One URL, automatic routing, OpenAI-compatible
Use a persistent library voice (`ref_voice: "my_narrator"`)	**:8080**	Only the orchestrator resolves library names; backends only accept raw `ref_audio`
OpenAI SDK drop-in	**:8080**	Already wired as `/v1`
Write batch scripts that only use preset voices	`:8091` directly	Skips the ~1 ms orchestrator hop; also lets you use vLLM-Omni's batch client utilities
Heavy-throughput cloning with in-memory reference caching	`:8093` directly	vLLM-Omni caches the speaker embedding server-side; repeated `ref_audio` uploads hit that cache
Debug which backend is misbehaving	Direct port	Isolates orchestrator bugs from model bugs
Benchmark pure model latency	Direct port	Removes orchestrator layer from the measurement
Ship only one task type (e.g. just preset voices)	`:8091` directly + no orchestrator	Smallest moving parts
Use features not yet in orchestrator (speaker-embedding interpolation, etc.)	Direct port	The orchestrator only proxies the standard speech endpoint

Rule of thumb: start with :8080. Drop down to a specific backend only when you have a concrete reason.

9.2 What each backend exposes (the per-port spec)

Every backend speaks the same three endpoints — they diverge only in what they accept inside `POST /v1/audio/speech`.

Endpoint	All 3 backends
`POST /v1/audio/speech`	Main synthesis (blocking or streaming)
`GET /v1/audio/voices`	List voices recognised by this backend
`POST /v1/audio/voices`	Upload a reference voice (only meaningful on Base)
`WS /v1/audio/speech/stream`	Incremental text input, PCM audio output

What differs per backend:

9.2.1 CustomVoice — port 8091

- **Model**: `Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice`
- **What it does**: synthesises speech with one of 9 predefined speaker identities.
- **Required fields**: `input`, `voice` (one of the 9 presets).
- **Optional fields**: `language`, `instructions`, `response_format`, `stream`, `speed`.
- **Ignored**: `ref_audio`, `ref_text`, `x_vector_only_mode` (these are Base-only).
- **Built-in voices**: `vivian`, `serena`, `uncle_fu`, `dylan`, `eric`, `ryan`, `aiden`, `ono_anna`, `sohee`.

```
# Preset voice + emotion, direct to :8091
curl -sX POST http://127.0.0.1:8091/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "input": "Direct-to-backend request, skipping the orchestrator.",
    "task_type": "CustomVoice",
    "voice": "ryan",
    "language": "English",
    "instructions": "Speak confidently and clearly",
    "response_format": "wav"
  }' --output direct_customvoice.wav

# List the 9 preset voices this server knows
curl -s http://127.0.0.1:8091/v1/audio/voices | jq .voices
```

9.2.2 VoiceDesign — port 8092

- **Model**: `Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign`
- **What it does**: designs a fresh voice from a natural-language description and synthesises the input text in it.
- **Required fields**: `input`, `instructions` (the voice description).
- **Optional fields**: `language`, `response_format`, `stream`, `speed`.
- **Ignored**: `voice`, `ref_audio`, `ref_text` (no predefined speakers, no reference cloning).
- **Built-in voices**: none — every call is a fresh voice.

```
# VoiceDesign, direct to :8092
curl -sX POST http://127.0.0.1:8092/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "input": "The first person on Mars opened the airlock and felt her boots crunch against red sand.",
    "task_type": "VoiceDesign",
    "language": "English",
    "instructions": "a calm female narrator in her late thirties, cinematic documentary tone",
    "response_format": "wav"
  }' --output direct_voicedesign.wav
```

9.2.3 Base — port 8093

- **Model**: `Qwen/Qwen3-TTS-12Hz-1.7B-Base`

- **What it does**: clones a voice from a reference audio clip (plus an optional transcript for ICL mode).
- **Required fields**: `input`, **one of** `ref_audio` (URL/data-URL) **or** an uploaded voice name (backend-side, via `POST /v1/audio/voices`).
- **Optional fields**: `ref_text` (strongly recommended), `x_vector_only_mode`, `language`, `instructions`, `response_format`, `stream`.
- **Ignored**: `voice` (in the preset sense — speakers are defined by the reference audio, not by name).
- **Built-in voices**: none — speakers come from the references you upload.

```
# Voice clone with inline base64 reference, direct to :8093
B64=$(base64 -w0 my_reference.wav)
curl -sX POST http://127.0.0.1:8093/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d "{
    \"input\": \"This is what my cloned voice now sounds like.\",
    \"task_type\": \"Base\",
    \"ref_audio\": \"data:audio/wav;base64,$B64\",
    \"ref_text\": \"The exact words from the reference clip.\",
    \"language\": \"English\",
    \"response_format\": \"wav\"
  }" --output direct_base.wav

# Or: upload once, reuse many times — via the backend's own voice upload endpoint
curl -sX POST http://127.0.0.1:8093/v1/audio/voices \
  -F "audio_sample=@my_reference.wav" \
  -F "consent=direct-$(date +%s)" \
  -F "name=narrator_v1" \
  -F "ref_text=The exact words from the reference clip." \
  -F "speaker_description=warm narrator"

# Subsequent clones reference that uploaded voice by name
curl -sX POST http://127.0.0.1:8093/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "input": "Same voice, new sentence.",
    "task_type": "Base",
    "voice": "narrator_v1",
    "response_format": "wav"
  }' --output direct_base_named.wav
```

> **Important**: voices uploaded *directly* to the Base backend on :8093 are **in-memory only**. They disappear on backend restart. The orchestrator's voice library on :8080 persists them to disk and re-uploads on restart — that's the main reason to use the orchestrator path for anything you care about keeping.

9.3 Use-case → which server, side-by-side

Same three tasks, once through the orchestrator and once direct, so you can see the difference.

Use case	Via orchestrator (:8080)	Direct to backend
Preset English voice	`POST :8080/v1/audio/speech` with `task_type: "CustomVoice"`	`POST :8091/v1/audio/speech` with no task routing, same body
Quick one-off voice design	`POST :8080/v1/audio/speech` with `task_type: "VoiceDesign"`	`POST :8092/v1/audio/speech`
Clone from a URL	`POST :8080/v1/audio/speech` with `ref_audio: "https://…"`	`POST :8093/v1/audio/speech` with the same body
Clone using a voice saved in the library	`POST :8080/v1/audio/speech` with `ref_voice: "my_narrator"` ✓	Not supported directly — backends don't know about the library. Use orchestrator.
Upload a voice so it survives restarts	`POST :8080/v1/audio/voices` (written to disk + mirrored to backend)	`POST :8093/v1/audio/voices` (in-memory only, lost on restart)
Stream PCM to a browser	`POST :8080/v1/audio/speech` with `stream: true, response_format: "pcm"`	Same body at `POST :8091..8093/v1/audio/speech`
Batch-generate 1 000 utterances	Either works, but scripting against direct ports avoids a copy step	

9.4 Minimal Python wrapper for direct calls

```
import httpx, base64, pathlib

PORTS = {"CustomVoice": 8091, "VoiceDesign": 8092, "Base": 8093}

def synth_direct(task_type, text, *, voice=None, instructions=None,
                 language="Auto", ref_wav_path=None, ref_text=None,
                 out="out.wav"):
    body = {"input": text, "task_type": task_type, "language": language,
            "response_format": "wav"}
    if voice:        body["voice"] = voice
    if instructions: body["instructions"] = instructions
    if ref_wav_path:
        b64 = base64.b64encode(pathlib.Path(ref_wav_path).read_bytes()).decode()
        body["ref_audio"] = f"data:audio/wav;base64,{b64}"
    if ref_text:     body["ref_text"] = ref_text

    r = httpx.post(f"http://127.0.0.1:{PORTS[task_type]}/v1/audio/speech",
                   json=body, timeout=300)
    r.raise_for_status()
    pathlib.Path(out).write_bytes(r.content)
    return out

# Examples
synth_direct("CustomVoice", "Hello.", voice="ryan",
             language="English", out="a.wav")
synth_direct("VoiceDesign", "Hello.",
             instructions="deep, theatrical male narrator",
             language="English", out="b.wav")
synth_direct("Base", "Hello.",
             ref_wav_path="my_reference.wav",
             ref_text="Transcript of reference.",
             language="English", out="c.wav")
```

9.5 When to skip the orchestrator entirely

If all three of these are true, deleting the orchestrator is fine and you save a process:

1. You only need **one** task type.
2. You don't need a UI.
3. You're okay with in-memory-only uploaded voices (Base) or no cloning at all.

In that case, disable the other programs in `configs/supervisord.conf.tpl` (set `autostart=false`) and point your client at the one backend port you actually use. The orchestrator becomes pure overhead.

But usually you don't even need to touch supervisord — the launcher scripts in the next section handle this for you.

11. Standalone single-model mode · integration guide

Most of the time you want the full stack — the orchestrator at `:8080` gives you task routing, voice-library persistence, the UI, and the OpenAI-compatible schema. But when you're **embedding Qwen3-TTS in another application** (a server, an agent, a pipeline), you usually just need **one task type** running on a single port that you can call from code. This section is the end-to-end recipe for that mode.

11.1 Launch a single backend (no orchestrator, no UI)

Three convenience scripts, each wrapping the generic `scripts/run_standalone.sh`:

```
bash scripts/run_customvoice.sh                 # preset speakers  :8091
bash scripts/run_voicedesign.sh                 # voice design     :8092
bash scripts/run_base.sh                        # voice cloning    :8093

# Or the generic form:
bash scripts/run_standalone.sh CustomVoice           # default :8091 on 0.0.0.0
bash scripts/run_standalone.sh VoiceDesign 8100      # pick another port
bash scripts/run_standalone.sh Base 8200 127.0.0.1   # localhost-only bind
bash scripts/run_standalone.sh --help                # full usage
```

What the script does:

1. Verifies the venv + chosen model's weights exist.
2. Checks the chosen port is free — if the full stack is already running on that port, it bails with a clear error.
3. Activates the venv and exec's `vllm-omni serve` in the foreground with the correct model directory + `configs/qwen3_tts_dgx.yaml`.
4. Prints a banner with the exact endpoint URL + a ready-to-paste curl example.
5. Traps `SIGINT` / `SIGTERM` for a clean shutdown when you `Ctrl-C`.

First launch takes ~30–90 s (weight load + CUDA-graph capture). Wait for `Uvicorn running on http://<host>:<port>` in the server output — then you're live.

Stop with `Ctrl-C` (or `kill <pid>`). No supervisord, no state to reconcile, no orphan processes.

11.2 Standalone vs full stack — which to use

Situation	Standalone	Full stack
Building a web app that calls TTS	✅ point at `:8091`	✅ point at `:8080`
Need persistent cloned-voice library across restarts	❌ (upload is in-memory only)	✅ orchestrator replays on boot
Need all three task types available at once	❌ run the full stack	✅
Need the browser UI	❌	✅
Need `ref_voice: "saved_name"` shortcuts	❌ — send `ref_audio` inline	✅
Containerising into your own deployment	✅ smallest footprint	—
Sharing the GPU with another workload	✅ easier to stop	✅ admin API is cleaner
CI / batch job worker	✅ spawn, call, terminate	—

11.3 Endpoints the standalone backend exposes

Each backend is a plain vLLM-Omni process speaking the OpenAI-compatible speech API. All endpoints below are served on whichever port you chose.

Method	Path	Purpose
`POST`	`/v1/audio/speech`	Synthesise speech — see §11.4 for body schema
`GET`	`/v1/audio/voices`	List built-in speakers + any voices uploaded to this backend process
`POST`	`/v1/audio/voices`	Upload a reference voice (only meaningful on Base; **in-memory only** — lost on restart)
`WS`	`/v1/audio/speech/stream`	WebSocket streaming text-in, per-sentence audio-out

11.4 Request schema

The JSON body for `POST /v1/audio/speech` is identical at `:8080` (orchestrator) and at `:8091 / 8092 / 8093` (standalone backends).

Field	Type	Per task	Notes
`input`	string	**required** all	The text to speak (≤ 8 000 chars)
`task_type`	`"CustomVoice"` / `"VoiceDesign"` / `"Base"`	required	Must match the backend you're hitting
`voice`	string	required for **CustomVoice**	`vivian`, `serena`, `uncle_fu`, `dylan`, `eric`, `ryan`, `aiden`, `ono_anna`, `sohee`
`instructions`	string	required for **VoiceDesign**, optional elsewhere	Free-form style / emotion / description
`ref_audio`	string	required for **Base**	HTTP URL, `file://…` URI, or `data:audio/wav;base64,…` — the reference clip
`ref_text`	string	optional but recommended for **Base**	Exact transcript of `ref_audio` — enables ICL mode (big quality win)
`x_vector_only_mode`	bool	optional **Base**	Skip the transcript path, use only speaker embedding. Faster, slightly lower fidelity
`language`	string	optional all	`Auto` (default), `English`, `Chinese`, `Japanese`, `Korean`, `German`, `French`, `Russian`, `Portuguese`, `Spanish`, `Italian`
`response_format`	string	optional all	`wav` (default), `mp3`, `flac`, `pcm`, `aac`, `opus`
`speed`	float	optional all	0.25 – 4.0. Ignored when `stream: true`
`stream`	bool	optional all	`true` requires `response_format: "pcm"`. Chunks of raw 16-bit PCM @ 24 kHz mono arrive as each Code2Wav window completes
`max_new_tokens`	int	optional all	Cap on codec tokens. Default 2048

Response is binary audio with matching `Content-Type` (e.g. `audio/wav`). Errors are JSON: `{"detail": "<message>"}`.

Error codes

HTTP	Meaning
`200`	Success; response body is the audio
`400`	Malformed body (missing required field, incompatible combo like `stream=true` + `response_format=wav`)
`404`	Unknown voice or library name (orchestrator only)
`413`	Voice upload > `MAX_UPLOAD_MB` (default 10 MB)
`500`	Internal error — check backend logs
`503`	Backend not ready / model not loaded

11.5 Python — sync client

```
import httpx, pathlib

BASE = "http://127.0.0.1:8091"  # standalone CustomVoice backend

def speak(text, voice="ryan", language="English", instructions=None, out="out.wav"):
    body = {
        "task_type": "CustomVoice",
        "voice": voice,
        "language": language,
        "input": text,
    }
    if instructions:
        body["instructions"] = instructions
    r = httpx.post(f"{BASE}/v1/audio/speech", json=body, timeout=300)
    r.raise_for_status()
    pathlib.Path(out).write_bytes(r.content)
    return out

speak("Hello from a Python script.", out="hello.wav")
```

11.6 Python — async streaming PCM (real-time playback)

```
import asyncio, httpx, sounddevice as sd

async def stream_tts(text, voice="vivian", base="http://127.0.0.1:8091"):
    async with httpx.AsyncClient(timeout=None) as client:
        async with client.stream(
            "POST", f"{base}/v1/audio/speech",
            json={
                "task_type": "CustomVoice", "voice": voice, "input": text,
                "stream": True, "response_format": "pcm",
            },
        ) as r:
            r.raise_for_status()
            tail = b""
            with sd.RawOutputStream(samplerate=24000, channels=1, dtype="int16") as out:
                async for chunk in r.aiter_bytes():
                    buf = tail + chunk
                    even = len(buf) & ~1
                    out.write(buf[:even])
                    tail = buf[even:]

asyncio.run(stream_tts("Streaming into my speakers in real time."))
```

11.7 Node.js — fetch

```
// save as speak.mjs, run: node speak.mjs
import { writeFile } from "node:fs/promises";

const BASE = "http://127.0.0.1:8092"; // standalone VoiceDesign backend

const r = await fetch(`${BASE}/v1/audio/speech`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    task_type: "VoiceDesign",
    language: "English",
    instructions: "a warm elderly narrator, slow and reassuring",
    input: "Once upon a time, deep in a quiet forest…",
  }),
});
if (!r.ok) throw new Error(`${r.status} ${await r.text()}`);
await writeFile("narrator.wav", Buffer.from(await r.arrayBuffer()));
console.log("wrote narrator.wav");
```

11.8 OpenAI SDK drop-in

The endpoint speaks the OpenAI speech API, so existing OpenAI clients work with zero code changes:

```
from openai import OpenAI

client = OpenAI(base_url="http://127.0.0.1:8091/v1", api_key="none")

r = client.audio.speech.create(
    model="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice",
    voice="aiden",
    input="The OpenAI SDK talks to our backend unchanged.",
)
r.stream_to_file("sdk_hello.wav")
```

```
// Node with openai@4+
import OpenAI from "openai";
import { writeFile } from "node:fs/promises";

const client = new OpenAI({ baseURL: "http://127.0.0.1:8091/v1", apiKey: "none" });
const r = await client.audio.speech.create({
  model: "Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice",
  voice: "ryan",
  input: "Works the same from Node.",
});
await writeFile("node_sdk.wav", Buffer.from(await r.arrayBuffer()));
```

11.9 curl — all three tasks

```
# Preset voice → WAV
curl -sX POST http://127.0.0.1:8091/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"CustomVoice","voice":"ryan","language":"English",
       "input":"Hello from curl."}' \
  --output hello.wav

# Voice design → WAV
curl -sX POST http://127.0.0.1:8092/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"VoiceDesign","language":"English",
       "instructions":"a crisp female news-reader, BBC style",
       "input":"Tonight: an unusual visitor in Hyde Park."}' \
  --output news.wav

# Voice clone with inline reference
B64=$(base64 -w0 my_reference.wav)
curl -sX POST http://127.0.0.1:8093/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d "{\"task_type\":\"Base\",
       \"input\":\"Cloned voice speaking.\",
       \"language\":\"English\",
       \"ref_audio\":\"data:audio/wav;base64,$B64\",
       \"ref_text\":\"What the reference clip says.\"}" \
  --output cloned.wav

# Streaming PCM to sox for real-time playback
curl -sX POST http://127.0.0.1:8091/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{"task_type":"CustomVoice","voice":"aiden","language":"English",
       "input":"Streaming mode.","stream":true,"response_format":"pcm"}' \
  --no-buffer | play -t raw -r 24000 -e signed -b 16 -c 1 -
```

11.10 Deployment patterns

Pattern A — Internal service (same host). Run `scripts/run_customvoice.sh` as a systemd unit, bind to `127.0.0.1:8091`, and call it from your app over loopback. No auth needed (localhost only). Model stays hot between requests.

```
# /etc/systemd/system/qwentts-customvoice.service
[Unit]
Description=Qwen3-TTS standalone CustomVoice backend
After=network.target

[Service]
WorkingDirectory=/path/to/Qwen-TTS-Studio
ExecStart=/path/to/Qwen-TTS-Studio/scripts/run_customvoice.sh
Restart=on-failure
RestartSec=10
User=<your-unix-user>

[Install]
WantedBy=multi-user.target
```

Then: `sudo systemctl enable --now qwentts-customvoice`.

Pattern B — Reverse-proxy (public endpoint). Put Nginx / Caddy / Traefik in front, terminate TLS there, add Bearer-token or mTLS auth, and proxy `/tts/` to `http://127.0.0.1:8091/`. Rate-limit at the proxy. The backend itself is unauthenticated.

Pattern C — Multi-tenant router. Launch each task type on its own port, and put a tiny router (FastAPI / Express / Fastify) in front that picks the backend by `task_type`. That's exactly what our full-stack orchestrator at `:8080` does; if you want that pattern without the UI / voice-library, our `orchestrator/` directory is the reference implementation (≈ 500 LOC Python).

Pattern D — Ephemeral job worker. CI / batch jobs spawn `run_standalone.sh`, call `/v1/audio/speech` a few times, then `SIGTERM` when done. The ~60 s cold-start cost amortises across the batch.

11.11 Production tips

- **Keep the backend warm** for interactive UX — cold start is ~30–90 s (weight load + CUDA-graph compile).
- **One model per process** — vLLM binds one model per instance; that's a framework constraint.
- **Budget ~6 GB GPU** per backend (≈ 3.5 GB weights + ~1 GB KV cache + ~1 GB vLLM overhead + headroom). On a 128 GB DGX Spark, all three together is trivial.
- **Prefer `stream=true` + `response_format="pcm"`** for sub-second first-audio. Your UI can start playback while the tail is still decoding.
- **Uploaded voices at the backend are in-memory only.** If you need persistence across restarts, either use the full-stack orchestrator, or replay uploads from your own store on startup.
- **CORS** is not set by vLLM-Omni. If you call from a browser at a different origin, reverse-proxy through Nginx / Caddy and add headers there.

12. Use-case recipes

9.1 Preset voice, English

```
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "task_type": "CustomVoice",
    "voice": "ryan",
    "language": "English",
    "input": "Hello from the DGX Spark — multiple models, one API."
  }' --output preset_ryan.wav
```

9.2 Preset voice with emotion and British accent

```
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "task_type": "CustomVoice",
    "voice": "aiden",
    "language": "English",
    "instructions": "Speak with a British accent, slow and conspiratorial",
    "input": "I have a feeling we shall find the answer — in the least expected place."
  }' --output preset_british.wav
```

9.3 Chinese (dialect) preset

```
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "task_type": "CustomVoice",
    "voice": "dylan",
    "language": "Chinese",
    "input": "今天天气真好，咱们一起去公园遛弯儿。"
  }' --output dylan_beijing.wav
```

9.4 Voice design — narrator

```
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "task_type": "VoiceDesign",
    "language": "English",
    "instructions": "a warm elderly male narrator with a slow, reassuring pace",
    "input": "Once upon a time, deep in a quiet forest, there lived a fox who believed she could hear t
  }' --output narrator.wav
```

9.5 Voice cloning — ad-hoc, with transcript (highest quality)

Reference audio as base64 data URL:

```
B64=$(base64 -w0 my_reference.wav)
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d "{
    \"task_type\": \"Base\",
    \"language\": \"English\",
    \"input\": \"This is my cloned voice, saying something I never recorded.\",
    \"ref_audio\": \"data:audio/wav;base64,$B64\",
    \"ref_text\": \"The exact words I said in the reference clip.\"
  }" --output cloned.wav
```

9.6 Voice cloning — persistent library voice

```
# 1. Upload once
curl -sX POST http://127.0.0.1:8080/v1/audio/voices \
  -F "audio_sample=@my_reference.wav" \
  -F "consent=ui-$(date +%s)" \
  -F "name=my_narrator" \
  -F "ref_text=The exact transcript of the reference clip." \
  -F "speaker_description=warm calm narrator" \
  -F "language=English"

# 2. Use arbitrarily many times
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "task_type": "Base",
    "language": "English",
    "input": "Chapter one. It was a bright cold day in April…",
    "ref_voice": "my_narrator"
  }' --output ch1.wav
```

9.7 Streaming PCM to speakers from the shell

```
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d '{
    "task_type": "CustomVoice",
    "voice": "vivian",
    "input": "Streaming straight into sox for real-time playback.",
    "stream": true,
    "response_format": "pcm"
  }' --no-buffer | play -t raw -r 24000 -e signed -b 16 -c 1 -
```

9.8 Python (httpx, non-streaming)

```
import httpx

r = httpx.post(
    "http://127.0.0.1:8080/v1/audio/speech",
    json={
        "task_type": "CustomVoice",
        "voice": "vivian",
        "language": "English",
        "input": "Hello from Python.",
    },
    timeout=300,
)
with open("out.wav", "wb") as f:
    f.write(r.content)
```

9.9 Python (async streaming PCM)

```
import asyncio, httpx, numpy as np, sounddevice as sd

async def stream_tts(text, voice="vivian"):
    async with httpx.AsyncClient(timeout=None) as client:
        async with client.stream(
            "POST",
            "http://127.0.0.1:8080/v1/audio/speech",
            json={
                "task_type": "CustomVoice", "voice": voice, "input": text,
                "stream": True, "response_format": "pcm",
            },
        ) as r:
            tail = b""
            with sd.RawOutputStream(samplerate=24000, channels=1, dtype="int16") as out:
                async for chunk in r.aiter_bytes():
                    buf = tail + chunk
                    even = len(buf) & ~1
                    out.write(buf[:even])
                    tail = buf[even:]

asyncio.run(stream_tts("Hello, world."))
```

9.10 OpenAI SDK

```
from openai import OpenAI
client = OpenAI(base_url="http://127.0.0.1:8080/v1", api_key="none")

r = client.audio.speech.create(
    model="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice",
    voice="ryan",
    input="Dropping into the OpenAI SDK works unchanged."
)
r.stream_to_file("sdk.wav")
```

9.11 Long-form narration

Pipe a full short story into the Design-voice task, language=Auto, response_format=mp3 for a smaller file:

```
curl -sX POST http://127.0.0.1:8080/v1/audio/speech \
  -H 'Content-Type: application/json' \
  -d "$(jq -Rn --rawfile text story.txt \
    '{task_type:"VoiceDesign",language:"Auto",
      instructions:"a warm elderly male narrator",
      input:$text, response_format:"mp3",
      max_new_tokens:8192}')" \
  --output story.mp3
```

9.12 LLM → TTS pipeline (WebSocket streaming)

When an upstream LLM is generating text token-by-token, use the WebSocket endpoint so each sentence can start synthesising as soon as it's complete:

```
import asyncio, json
import websockets

async def llm_to_tts(tokens):
    async with websockets.connect("ws://127.0.0.1:8080/v1/audio/speech/stream") as ws:
        await ws.send(json.dumps({
            "type": "session.config",
            "voice": "ryan", "task_type": "CustomVoice",
            "language": "English", "stream_audio": True,
            "response_format": "pcm",
        }))
        # As each token arrives from your LLM:
        async for tok in tokens:
            await ws.send(json.dumps({"type": "input.text", "text": tok}))
        await ws.send(json.dumps({"type": "input.done"}))
        async for msg in ws:
            if isinstance(msg, bytes):
                ...  # feed to your audio device
            else:
                evt = json.loads(msg)
                if evt.get("type") == "session.done":
                    break
```

13. Troubleshooting

Symptom	Likely cause	Fix
`flash-attn` build fails after ~20 min	Memory pressure at peak of nvcc compile	Re-run `scripts/00_install.sh` with `MAX_JOBS=2 NVCC_THREADS=2`
`fa3-fwd` install error	Stale clone of `vllm-omni/requirements/cuda.txt`	`rm -rf vllm-omni && bash scripts/00_install.sh`
Backend process `FATAL` in supervisorctl	OOM on model load	Check `logs/<backend>.log`. Lower `gpu_memory_utilization` in `configs/qwen3_tts_dgx.yaml` and restart
`/health` stays `degraded` indefinitely	A backend failed to load	`bash scripts/status.sh` and `tail -50 logs/<backend>.log` — the traceback will say
`503 <task> backend is down` on generate	Backend crashed or still loading	Wait 60 s; if still down, `bash scripts/restart.sh` and check its log
`400 Unsupported speaker`	Wrong `voice` for the backend	`GET /v1/audio/voices` — use a name from the right `builtin_by_task` list
`400 VoiceDesign task requires 'instructions'`	Empty `instructions` for VoiceDesign	That field is required — describe the voice
`400 Base task requires 'ref_audio' or 'ref_voice'`	No reference supplied for cloning	Attach a file, data URL, or `ref_voice: <library-name>`
Audio has clicks at chunk boundaries in UI	Browser dropped a chunk	Toggle "Stream as generated" off once to let the full WAV play
First-token latency feels slow	First request after startup warms CUDA graphs	Second request should be 2–3× faster
Port already in use	Another `supervisord` session left behind	`pkill supervisord` then `bash scripts/start.sh`

Where to look when stuck:

- `logs/install.log` — any install error you don't remember.
- `logs/<backend>.log` — the full vLLM-Omni stack trace.
- `logs/orchestrator.log` — routing errors, upload errors, backend probe failures.
- `logs/supervisord.log` — process starts/stops.

14. Operating guide

11.1 Changing ports

Edit `.env`, restart. All four services read their ports from it via `envsubst` at start time.

11.2 Changing the GPU memory split

If you see OOM on model load, lower `STAGE_GPU_MEM_UTIL` in `.env` and restart. 0.15 is the safe default with all three models resident; you can try 0.10 at the cost of a smaller KV cache.

11.3 Idling a model out

If you want to run only two models instead of three (e.g. to save a little memory), edit `configs/supervisord.conf.tpl` and set `autostart=false` on the program you want to idle. Start it manually with `supervisorctl start vllm_voicedesign` when needed.

11.4 Adding a new cloned voice programmatically

```
import httpx
files = {"audio_sample": ("me.wav", open("me.wav","rb"), "audio/wav")}
data  = {
    "consent": "ui-12345",
    "name": "my_voice",
    "ref_text": "This is exactly what I said.",
    "speaker_description": "calm, mid-40s",
    "language": "English",
}
r = httpx.post("http://127.0.0.1:8080/v1/audio/voices", files=files, data=data)
print(r.json())
```

11.5 Clearing the voice library

```
curl -s http://127.0.0.1:8080/v1/audio/voices | jq -r '.uploaded_voices[].name' \
  | while read name; do
      curl -sX DELETE "http://127.0.0.1:8080/v1/audio/voices/$name" >/dev/null
    done
```

Or delete `data/voices/` and `data/voices.json` manually (with the server stopped).

15. What's under the hood

15.1 The Qwen3-TTS architecture (abbreviated)

```
text  →  Qwen2 BPE  →  Talker (28-layer Transformer + MRoPE + SWA)  →  codebook-0 stream
                                  │
                                  └→  CodePredictor (5 layers) + 15 heads  →  codebooks 1…15
                                                                                  │
                                                                                  ▼
                                                          SpeechTokenizer-12Hz decoder
                                                          (fully causal ConvNet, SplitRVQ 1+15)
                                                                                  │
                                                                                  ▼
                                                                   24 kHz mono PCM waveform
```

- **Talker**: autoregressive decoder, predicts the semantic codebook at 12.5 frames/sec.
- **CodePredictor + 15 heads**: in one forward per frame, predicts the 15 residual acoustic codebooks.
- **SpeechTokenizer decoder**: deterministic causal ConvNet (no diffusion, no flow-matching, no BigVGAN), upsamples 12 Hz codec frames to 24 kHz audio via transposed convs and ConvNeXt blocks.

That lightweight, fully-causal decoder is why 12Hz Qwen3-TTS achieves sub-100 ms first-packet latency on Hopper reference hardware.

15.2 The vLLM-Omni pipeline

Each of our three backends runs a two-stage pipeline:

1. **Stage 0 — Talker** (`max_num_seqs=4`, CUDA graphs enabled). Autoregressive LM decode. Emits per-frame codes.
2. **Stage 1 — Code2Wav decoder** (`max_num_seqs=1`, enforce_eager). Takes codes from Stage 0 via a shared-memory connector, runs the codec decoder, emits raw waveform chunks.

`async_chunk: true` makes Stage 1 emit PCM frames as Stage 0 produces codes — this is what enables true real-time streaming in the `stream=true` path.

15.3 Why three servers, not one

vLLM binds **one model per server instance**. Since we want all three task types always-on for zero-switch latency, we run three servers on three internal ports (8091/8092/8093). The orchestrator is the only public-facing port. This is consistent with how vLLM-Omni itself recommends deploying TTS in production (see the vllm-omni examples under `examples/online_serving/qwen3_tts/`).

16. Environment variables reference (`.env`)

Variable	Default	Meaning
`ROOT_DIR`	absolute path to your clone	Absolute root of the project (set via `cp .env.example .env` and edit)
`MODELS_DIR`	`$ROOT_DIR/models`	Where weight directories live
`DATA_DIR`	`$ROOT_DIR/data`	Voice library + generated cache
`LOGS_DIR`	`$ROOT_DIR/logs`	Process logs
`ORCH_HOST`, `ORCH_PORT`	`127.0.0.1:8080`	Public orchestrator address
`CUSTOMVOICE_HOST`, `CUSTOMVOICE_PORT`	`127.0.0.1:8091`	Internal, CustomVoice backend
`VOICEDESIGN_HOST`, `VOICEDESIGN_PORT`	`127.0.0.1:8092`	Internal, VoiceDesign backend
`BASE_HOST`, `BASE_PORT`	`127.0.0.1:8093`	Internal, Base (clone) backend
`CUSTOMVOICE_MODEL_PATH` etc.	`$MODELS_DIR/Qwen3-TTS-...`	Absolute model directory each backend loads
`STAGE_GPU_MEM_UTIL`	`0.15`	Per-stage cap. 6 stages × 0.15 = 0.9 total utilisation
`DEFAULT_STREAM`	`true`	UI default for the stream toggle
`DEFAULT_RESPONSE_FORMAT`	`wav`	UI default download format
`DEFAULT_SAMPLE_RATE`	`24000`	Reported in stream response headers
`MAX_UPLOAD_MB`	`10`	Voice upload size cap
`VOICES_JSON`, `VOICES_DIR`	under `data/`	Paths for the voice library store

17. Known limitations (v0.1.0)

- **Single-request serving.** Batch processing is not yet optimised for online serving in vLLM-Omni; a burst of concurrent requests may queue. Serial latency is the metric that's tuned.
- **Localhost binding only.** If you want to expose this on a LAN, change `ORCH_HOST=0.0.0.0` in `.env` and add a reverse proxy with auth in front of it. There is no auth in-proc.
- **CPU audio I/O.** Reference-audio decoding uses `librosa` on the CPU for now. Unlikely to matter at single-stream rates but would show up under heavy cloning load.
- **No fp8 / NVFP4 yet.** Models run in bf16. NVFP4 on Blackwell would roughly halve model bandwidth and should give a real speedup; support lands in vLLM-Omni when the Qwen3-TTS 16-codebook head gets a TRT-LLM plugin.

- **Flash-Attention v2.** v3 is Hopper-only; Blackwell users stay on FA2 for now.

---

18. Credits & references

- **Qwen3-TTS paper** — Alibaba Qwen team, "Qwen3-TTS Technical Report" (arXiv 2601.15621).
- **Qwen3-TTS weights & docs** — `github.com/QwenLM/Qwen3-TTS`, `huggingface.co/collections/Qwen/qwen3-tts`.
- **vLLM-Omni** — `github.com/vllm-project/vllm-omni`, especially `examples/online_serving/qwen3_tts/`.
- **DGX Spark aarch64 vLLM guide** — NVIDIA Developer Forums thread on running vLLM-Omni Qwen3-TTS on GB10.
- **Community forks** consulted for optimisation research — `andimarafioti/faster-qwen3-tts`, `rekuenkdr/Qwen3-TTS-streaming`, `dffdeeq/Qwen3-TTS-streaming`.

All Qwen3-TTS models are released under **Apache-2.0**. This project is a wrapper — no new weights are trained.

End of guide.